

LinglongAMG 26 Standard Edition User Guide

C API Usage and Reference

Document revision: 1.9

Language: English

Revision date: October 26, 2026

Contents

1 Introduction **1**
1.1 Supported Features . . . 1
1.2 Conventions . . . 1

2 Installation and Quick Start **2**
2.1 Package Layout . . . 2
2.2 Configure Runtime Paths and the License . . . 2
2.3 License Lifecycle . . . 3
2.4 Build and Run the Standard Sample . . . 3
2.5 Link an Application . . . 4
2.6 CMake Integration . . . 4

3 General Usage **5**
3.1 Handles and State . . . 5
3.2 Execution Phases . . . 5
3.3 Memory Ownership and Lifetime . . . 5
3.4 Devices and Concurrency . . . 5
3.5 JSON Configuration . . . 6
3.6 Reuse Conditions . . . 6
3.7 Single-Cycle Preconditioning . . . 6

4 Data Types and Storage Formats **7**
4.1 Opaque Objects . . . 7
4.2 Values and Indices . . . 7
4.3 CSR Storage . . . 7
4.4 Example Matrix . . . 7
4.5 Vectors and Solve Information . . . 8
4.6 Buffer Capacity . . . 8

5 API Reference **9**
5.1 `linglongamg_create_from_config` . . . 9
5.2 `linglongamg_read_csr` . . . 10
5.3 `linglongamg_setup` . . . 11
5.4 `linglongamg_solve` . . . 11
5.5 `linglongamg_get_solve_info` . . . 12
5.6 `linglongamg_reset` . . . 13
5.7 `linglongamg_destroy` . . . 13
5.8 `linglongamg_csr_destroy` . . . 14
5.9 `linglongamg_status_string` . . . 14

6 Examples **15**
6.1 General CBD Test Program . . . 15
6.2 Core Calling Sequence . . . 15
6.3 Successive RHS Vectors for One Matrix . . . 16
6.4 Apply One AMG Cycle . . . 16

7 Error Handling and Result Validation **17**

7.1 Status Codes . . . 17
7.2 Diagnosis and Recovery by Phase . . . 17
7.3 Common Integration Problems . . . 18
7.4 Numerical Accuracy and Residuals . . . 18
7.5 Finite Values and Display Precision . . . 18
7.6 Performance Measurement . . . 19
7.7 GPU Memory Planning . . . 19
7.8 Issue Reports . . . 19

Appendix A Example Input Files 20
A.1 CBD Binary Layout . . . 20
A.2 Sample RHS and Application Input . . . 20

Appendix B Documentation and Support 21
B.1 Package Documents . . . 21
B.2 Technical Support . . . 21

Appendix C Version History 22
C.1 Product Version . . . 22
C.2 Document Revisions . . . 22

1 Introduction

LinglongAMG is a GPU iterative library for sparse linear systems. Its public C API solves

$$Ax = b, \qquad A \in \mathbb{R}^{n \times n}, \quad b, x \in \mathbb{R}^n.$$

Here A is the coefficient matrix, b is the right-hand side (RHS), and x is the solution. Setup constructs the algebraic multigrid (AMG) hierarchy; Solve uses it in the configured iteration. An application can also apply one complete AMG cycle and control convergence with an external Krylov iteration.

1.1 Supported Features

Item	Support
Value type	Delivered ABI: real FP64 (`double`); no complex-input interface.
Sparse storage	Full, zero-based device CSR; signed 32-bit dimensions and indices.
RHS and solution	One RHS per call; contiguous device vectors of length n.
GPU use	One GPU per system; no public cooperative multi-GPU or distributed MPI interface.
Repeated solves	Reuse the hierarchy for an unchanged matrix and solve successive RHS vectors.
Memory scope	No automatic host-memory spill or out-of-core workflow.

1.2 Conventions

Monospaced text denotes functions, types, fields, and commands. Product version 26.43 identifies the runtime; document revision 1.9 identifies this guide. Consult the supplied header, README, configuration, and release information for capabilities and system, GPU, driver, and dependency requirements. Algorithm combinations may differ by platform.

> **Note: Applicability**
> Check data specifications and matrix properties, then validate convergence and resources. PCG requires appropriate symmetric positive-definite conditions. Applications handle projections or constraints for singular systems. AMG convergence is not guaranteed for arbitrary systems.

2 Installation and Quick Start

Obtain a package matching the GPU, system ABI, and runtime, together with a valid license. Examples link to the supplied shared library; rebuilding LinglongAMG is not required.

2.1 Package Layout

```
LinglongAMG/
|-- lib/liblinglongamg.so
|-- lib/linglongamg.h
|-- test_src/
|-- build_sample.sh
|-- CMakeLists.txt
|-- linglongamg_config.json
|-- spe10.cbd
|-- run_linglongamg.sh
`-- README.md
```

This layout includes the standard sample. Place the separately issued original `license.key` at the root, replacing any empty placeholder. Keep the library, header, and supporting files from the same build. A `build/` directory is generated locally rather than prebuilt for delivery.

2.2 Configure Runtime Paths and the License

Run these commands from the package parent directory. `LINGLONGAMG_SDK` is a shell variable used by this example to locate files, not a library authorization setting.

```
cd ./LinglongAMG
export LINGLONGAMG_SDK="$PWD"
export LD_LIBRARY_PATH="$LINGLONGAMG_SDK/lib${LD_LIBRARY_PATH:+:$LD_LIBRARY_PATH}"
test -s "$LINGLONGAMG_SDK/license.key"
```

The last command checks only that the file is nonempty, not that authorization is valid. Variables apply to this terminal and its child processes; configure batch environments too. Keep the quotes for paths containing spaces.

Use a valid original license file without manual edits. Authorization is checked locally; runtime internet access is not required. The current build discovers SDK-local `license.key` or `licenses/*.key`; do not apply another product's license-path environment variables.

2.3 License Lifecycle

Handle creation reads JSON and validates the product, device, version range, validity period, delivery integrity, and signed revocation-list (CRL) policy. Select the GPU through the platform runtime first; matrix and vectors must reside there. The constructor does not take a separate index into a list of licensed GPUs.

The licensed build checks authorization again and commits results in `linglongamg_get_solve_info`. Read x only after that call succeeds; on failure, old buffer contents are not the current result. Creation and post-solve query are outside Setup/Solve timing; see Section 7.

After updating a license or CRL, finish the current computation, destroy the old handle, and create a new one. Do not rely on automatic reloading by existing handles. Follow the delivery workflow when updating signed files; do not edit them yourself.

2.4 Build and Run the Standard Sample

```
chmod +x ./build_sample.sh ./run_linglongamg.sh
./build_sample.sh
./run_linglongamg.sh
./run_linglongamg.sh -A ./my_matrix.cbd -c ./my_config.json
```

Load the required platform toolchain and CMake first. Defaults are `spe10.cbd` and `linglongamg_config.json` at the package root; explicit relative paths use the current shell directory. The standard sample generates $b = A\mathbf{1}$ and uses $x_0 = 0$ for a known-solution check. Applications provide their business RHS through the API.

Retain API status, timings, and the original-equation residual. API success does not imply accuracy acceptance. See Section 7 for error codes, NaN, Inf, or unusual residuals.

2.5 Link an Application

The manual's `examples/cbd_sample.cpp` is a general CBD tester; `cbd_reader.h` validates input. Defaults are `spe10.cbd` and `linglongamg_config.json` in the working directory. It generates $b = A\mathbf{1}$, starts from zero, and verifies the result independently. Keep the complete runtime and license in the SDK, selected with `AMG_ROOT`. The header and library must match. The following layout places the example beside the SDK:

```
Parent/
|-- LinglongAMG/
|   |-- lib/liblinglongamg.so
|   |-- lib/linglongamg.h
|   |-- license.key
|   |-- spe10.cbd
|   `-- linglongamg_config.json
`-- examples/
    |-- cbd_sample.cpp
    |-- cbd_reader.h
    `-- CMakeLists.txt
```

2.6 CMake Integration

Run from examples with the matching development toolkit and runtime:

```
cmake -S . -B build -DBACKEND=CUDA -DAMG_ROOT=../LinglongAMG
cmake --build build --parallel
./build/cbd_sample -A ../LinglongAMG/spe10.cbd \
    -c ../LinglongAMG/linglongamg_config.json
```

If CUDA is not found, add `-DCUDAToolkit_ROOT="$CUDA_HOME"` when configuring. For Hygon, load the matching DTK environment and use:

```
cmake -S . -B build-hip -DBACKEND=HIP \
    -DAMG_ROOT=../LinglongAMG -DDTK_ROOT="$ROCM_PATH"
cmake --build build-hip --parallel
./build-hip/cbd_sample -A ../LinglongAMG/spe10.cbd \
    -c ../LinglongAMG/linglongamg_config.json
```

The full CMake file is supplied with the example. The build's runtime path points to the chosen SDK; reconfigure after moving directories or machines. Preserve the relative placement of license and library.

3 General Usage

3.1 Handles and State

The handle stores configuration, the AMG hierarchy, and workspace. Follow this sequence; stop dependent calls if any step fails.

```
select device; create_from_config(config)
upload CSR, b, x0; read_csr(...)
setup(solver, A)
solve(solver, b, x)
get_solve_info(solver, info)
read x; verify original-system residual
destroy(solver); csr_destroy(A); free device arrays
```

3.2 Execution Phases

Phase	Purpose
Creation	Read JSON, validate authorization, and create the handle.
Setup	Construct the hierarchy and workspace from CSR.
Solve	Run the configured solver or cycle; licensed results are staged.
Result query	Validate authorization, commit the solution, and return information.

3.3 Memory Ownership and Lifetime

The CSR descriptor references existing device arrays without copying or taking ownership. Applications allocate, transfer, and free data. Keep matrix, b, and x valid while in use, and do not consume x as the current result before a successful query. Destroy the solver first, then the CSR descriptor, and finally free device buffers.

3.4 Devices and Concurrency

Use a handle sequentially without concurrent or reentrant calls, and keep device selection unchanged throughout its lifetime. Manage handles and dependencies separately for independent systems; a device pointer cannot be used directly on another GPU.

3.5 JSON Configuration

`setup.max_levels`	Maximum hierarchy depth.
`setup.min_coarse_size`	Coarsest-level size threshold.
`setup.strong_threshold`	Strong-connection threshold.
`setup.presweeps` `setup.postsweeps`	Pre- and post-smoothing sweeps.
`solve.method`	Solver method or single-cycle mode.
`solve.smoother` `solve.cycle`	Level smoother and cycle type.
`solve.max_iters` `solve.tolerance`	Iteration limit and internal tolerance.
`solve.norm` `solve.convergence`	Norm and normalization convention.

Start with the supplied configuration, preserving each field's setup/solve object. The manual example uses GMRES, AMG preconditioning, a V-cycle, and L1-Jacobi; these are not universal platform defaults. Parsing an option does not imply backend support. Recreate the handle and run Setup after configuration changes.

3.6 Reuse Conditions

Change	Action
Only b changes	Keep the unchanged matrix and hierarchy, initialize x, solve and query again.
Values of A change	Re-run Setup; do not assume the old hierarchy remains valid.
Pattern or dimensions change	Recreate the descriptor and solver and run complete Setup.
After Reset	Run Setup before Solve; recreate the handle for configuration changes.

3.7 Single-Cycle Preconditioning

Use `method=amg` and `max_iters=1` in a compatible configuration. Supply the outer residual r, zero z, apply the complete cycle to obtain $z \approx M^{-1}r$, and return it after successful query. A very small internal tolerance suppresses early stopping; it is not the application acceptance tolerance.

4 Data Types and Storage Formats

4.1 Opaque Objects

```
typedef struct linglongamg_solver_st *linglongamg_handle_t;
typedef struct linglongamg_csr linglongamg_csr_t;
```

Applications retain pointers without accessing internal fields. Use the matching header and do not alter ABI declarations.

4.2 Values and Indices

The delivered `linglongamg_real_t` is `double`. Do not define `LINGLONGAMG_USE_FP32` while linking the same FP64 library. Dimensions, nnz, row pointers, and column indices use signed 32-bit int; check overflow and buffer capacity during conversion.

4.3 CSR Storage

`row_ptr`	$n+1$ entries, starts at 0, nondecreasing, ends at nnz.
`col_idx`	nnz entries in $[0, n)$; sort each row and merge duplicates beforehand.
`values`	nnz finite FP64 real entries in column-index order.

The matrix must be nonempty and square. Supply full CSR even for symmetric matrices, not just one triangle. All arrays reside on the same GPU; the application uploads host data first.

4.4 Example Matrix

$$A = \begin{bmatrix} 4 & -1 & 0 & 0 \\ -1 & 4 & -1 & 0 \\ 0 & -1 & 4 & -1 \\ 0 & 0 & -1 & 3 \end{bmatrix}, \quad b = \begin{bmatrix} 3 \\ 2 \\ 2 \\ 2 \end{bmatrix}, \quad x_* = \mathbf{1}.$$

```
row_ptr = [0, 2, 5, 8, 10]
col_idx = [0,1, 0,1,2, 1,2,3, 2,3]
values  = [4,-1, -1,4,-1, -1,4,-1, -1,3]
```

4.5 Vectors and Solve Information

```
typedef struct {
    int n;
    linglongamg_real_t *val;
} linglongamg_vec_t;

typedef struct {
    int converged;
    int iterations;
    linglongamg_real_t initial_residual;
    linglongamg_real_t final_residual;
    linglongamg_real_t relative_residual;
    double setup_ms;
    double solve_ms;
} linglongamg_solve_info_t;
```

The vector descriptor is on the host; val points to a device array. The RHS is input, while x holds the initial guess and the solution committed by the query. Allocate them separately to avoid aliasing.

`converged`	Internal stopping criterion, not independent acceptance.
`iterations`	Number of iterations performed.
`initial_residual` `final_residual`	Initial and final residual under the configured definition.
`relative_residual`	Relative residual normalized as configured.
`setup_ms` `solve_ms`	Internal timings in milliseconds.

4.6 Buffer Capacity

For FP64 and 32-bit indices, base CSR storage is $4(n+1)+12\,\mathrm{nnz}$ bytes; each vector uses $8n$ bytes. This excludes hierarchy matrices, interpolation, workspace, runtime allocations, and staged results. Measure peak GPU memory.

5 API Reference

This chapter lists the core functions declared in the public header `linglongamg.h`. Input parameters are supplied by the application; output parameters receive newly created objects or computation results. Handle, descriptor, and buffer lifetime rules are described in Chapter 3. Data types and storage formats are described in Chapter 4.

Function	Purpose
`linglongamg_create_from_config`	Load JSON, validate authorization, and create a handle.
`linglongamg_read_csr`	Describe application-owned CSR arrays.
`linglongamg_setup`	Build the AMG hierarchy and workspace.
`linglongamg_solve`	Compute a pending solution.
`linglongamg_get_solve_info`	Validate, publish x, and return solve information.
`linglongamg_reset`	Clear state and hierarchy for a new Setup.
`linglongamg_destroy`	Destroy the handle and internal resources.
`linglongamg_csr_destroy`	Destroy the descriptor; retain application arrays.
`linglongamg_status_string`	Return a read-only status description.

Except for `linglongamg_csr_destroy`, which returns void, and `linglongamg_status_string`, which returns a string, the functions above return `linglongamg_status_t`; success is `LINGLONGAMG_STATUS_SUCCESS`. Check every returned status. On failure, stop dependent operations and follow Chapter 7. Read or download the solution only after `linglongamg_get_solve_info` succeeds; old buffer contents must not be treated as the current result.

5.1 `linglongamg_create_from_config`

```
linglongamg_status_t linglongamg_create_from_config(
    linglongamg_handle_t *solver, const char *config_file);
```

Parameters

`solver`	Address receiving the output handle.
`config_file`	Readable JSON path; relative paths use the current process directory.

Behavior and Return

Call after selecting the GPU. Reads configuration and checks authorization. Setup requires success; stop subsequent calls if creation fails.

5.2 `linglongamg_read_csr`

```
linglongamg_status_t linglongamg_read_csr(
    linglongamg_csr_t **A, int nrows, int ncols, int nnz,
    const int *device_row_ptr, const int *device_col_idx,
    const linglongamg_real_t *device_values);
```

Parameters

`A`	Address receiving the CSR descriptor.
`nrows, ncols, nnz`	Square dimensions and stored-entry count, within int32 range and matching arrays.
`device_row_ptr`	Device row pointers, length nrows+1.
`device_col_idx`	Device column indices, length nnz.
`device_values`	Device FP64 values, length nnz.

Behavior and Return

Creates a descriptor referencing existing arrays. Does not read disk files, run Setup, or take ownership of input memory. Arrays must satisfy Section 4.

5.3 linglongamg_setup

```
linglongamg_status_t linglongamg_setup(
    linglongamg_handle_t solver, const linglongamg_csr_t *A);
```

Parameters

solver	Valid configured handle.
A	Descriptor referring to valid device CSR arrays.

Behavior and Return

Builds the AMG hierarchy and workspace. Solve requires success. Complete the previous result query first; Section 3 describes rebuilding after matrix changes.

5.4 linglongamg_solve

```
linglongamg_status_t linglongamg_solve(
    linglongamg_handle_t solver, const linglongamg_vec_t *b,
    linglongamg_vec_t *x);
```

Parameters

solver	Handle with successful Setup and no result awaiting query.
b	Host input descriptor whose val points to the RHS on the same GPU.
x	Host descriptor for initial guess and output buffer; same length as b.

Behavior and Return

Executes the configured solver or cycle. Success does not guarantee application accuracy; a licensed result still awaits query and commit. Successfully query after every Solve before reading the solution or solving again.

5.5 `linglongamg_get_solve_info`

```
linglongamg_status_t linglongamg_get_solve_info(
    linglongamg_handle_t solver, linglongamg_solve_info_t *info);
```

Parameters

`solver`	Handle whose current Solve returned success.
`info`	Writable address of a host information structure.

Behavior and Return

Checks post-solve authorization, commits the solution, and returns the structure in Section 4. Read or download x only after SUCCESS. Do not consume results on failure; Section 7 describes recovery.

5.6 `linglongamg_reset`

```
linglongamg_status_t linglongamg_reset(linglongamg_handle_t solver);
```

Parameters

`solver`	Valid handle that has not been destroyed.

Behavior and Return

Clears solve state and hierarchy workspace; run Setup before another Solve. Does not read new JSON or replace handle recreation after a license update. Prefer destruction and recreation for error recovery.

5.7 `linglongamg_destroy`

```
linglongamg_status_t linglongamg_destroy(linglongamg_handle_t solver);
```

Parameters

`solver`	Handle whose lifetime is ending.

Behavior and Return

Releases resources owned by the handle; do not access it afterward. The application must still destroy the CSR descriptor and free its own device arrays.

5.8 `linglongamg_csr_destroy`

```
void linglongamg_csr_destroy(linglongamg_csr_t *A);
```

Parameters

`A`	CSR descriptor no longer used by a solver.

Behavior and Return

Destroys the descriptor without freeing referenced device arrays. This function has no return status.

5.9 `linglongamg_status_string`

```
const char *linglongamg_status_string(linglongamg_status_t status);
```

Parameters

`status`	Status returned by a public API.

Behavior and Return

Returns a read-only diagnostic string; do not modify or free it. Use enumeration values for logic and retain original logs for diagnosis.

> **Note: Authorization diagnostics**
> Some headers also declare machine-identification and authorization-diagnostic functions. They do not replace validation at creation and query. Use the collector and activation workflow specified by the current package.

6 Examples

6.1 General CBD Test Program

The program reads the matrix and JSON configuration, links the shared library, and independently verifies the original-system residual. It supports CUDA and HIP builds. Input layout is specified in Appendix A. With no arguments, it reads spe10.cbd and `linglongamg_config.json` from the working directory.

```
./build/cbd_sample
./build/cbd_sample -A ./spe10.cbd -c ./linglongamg_config.json
./build/cbd_sample -A ./other.cbd -c ./config.json \
    --rtol 1e-8 --device 0
```

`--rtol` defaults to 10^{-6} independently of the JSON solver tolerance; `--device` defaults to 0. Configuration is not rewritten. It generates $b = A\mathbf{1}$ for a known-solution test, starts from zero, and does not read an application RHS file.

6.2 Core Calling Sequence

This excerpt shows the calls after data upload. Full option parsing, CBD reading, cleanup, and verification are supplied in the source package.

```
amg_check(linglongamg_create_from_config(&solver.handle,config));
amg_check(linglongamg_read_csr(&solver.matrix,a.n,a.n,a.nnz,
                               dr.ptr,dc.ptr,da.ptr));
linglongamg_vec_t rhs{a.n,db.ptr}, solution{a.n,dx.ptr};
gpu_check(GPU(DeviceSynchronize)()); auto t0=Clock::now();
amg_check(linglongamg_setup(solver.handle,solver.matrix));
gpu_check(GPU(DeviceSynchronize)()); auto t1=Clock::now();
amg_check(linglongamg_solve(solver.handle,&rhs,&solution));
gpu_check(GPU(DeviceSynchronize)()); auto t2=Clock::now();
linglongamg_solve_info_t info{};
amg_check(linglongamg_get_solve_info(solver.handle,&info));
gpu_check(GPU(DeviceSynchronize)()); auto t3=Clock::now();
// Download x only after the query succeeds.
```

Timings use two decimal places with ms/us units; residuals and tolerances use scientific notation with two decimal places. Independent verification uses long double accumulation and stable norms and checks that device RHS data is unchanged. Exit 0 means acceptance passed, 2 means residual acceptance failed, and 1 indicates an argument, file, GPU, API, or license error. For a zero RHS, the relative residual is undefined and acceptance requires a zero absolute residual.

6.3 Successive RHS Vectors for One Matrix

```
/* Keep A unchanged; run Setup once. */
for (int k = 0; k < rhs_count; ++k) {
    upload_rhs(k, b_device);  /* application operation */
    zero_solution(x_device); /* application operation */
    check(linglongamg_solve(solver, &b, &x));
    check(linglongamg_get_solve_info(solver, &info));
    consume_and_verify(k, x_device);
}
```

Upload, zeroing, checking, and verification functions are application pseudocode, not library APIs. Record each RHS status, time, and residual separately; an average must not conceal exceptional results.

6.4 Apply One AMG Cycle

```
/* Compatible configuration: method=amg, max_iters=1. */
zero_solution(z_device);
linglongamg_vec_t r = {n, residual_device};
linglongamg_vec_t z = {n, z_device};
check(linglongamg_solve(solver, &r, &z));
check(linglongamg_get_solve_info(solver, &info));
/* The outer iteration may now consume z_device. */
```

A nonconverged flag after one cycle can be expected. The outer algorithm controls overall convergence and application acceptance. Do not relax the application residual criterion just to change an internal flag.

7 Error Handling and Result Validation

API status indicates operation completion; numerical acceptance indicates solution quality. Record them separately.

7.1 Status Codes

0 / `SUCCESS`	Operation succeeded; independently verify the result.
1 / `INVALID_VALUE`	Invalid arguments, dimensions, or call state.
2 / `ALLOC_FAILED`	Allocation failed; check host and device memory.
3 / `CUDA_ERROR`	GPU operation failed; the enumeration retains the CUDA name.
4 / `NOT_IMPLEMENTED`	Requested feature not implemented.
5 / `RUNTIME_ERROR`	Runtime error; retain original diagnostics.
6 / `LICENSE_ERROR`	License, device, validity, or integrity validation failed.

Prefix enumerations with `LINGLONGAMG_STATUS_`. Internal negative diagnostics are not these public enumerations. Retain the first failing API, its return code, and the complete original log.

7.2 Diagnosis and Recovery by Phase

Creation	JSON path, original license, device selection, matching files, and permissions.
Descriptor / Setup	CSR dimensions, indices, sorting, memory location, configuration, and GPU memory.
Solve	Successful Setup, matching vectors and device, finite values, and previous result query.
Result query	Successful Solve and valid authorization; do not use a failed result.

Stop dependent calls after failure and clean up created resources. Correct the issue, recreate the handle, and run Setup. Old x contents are not the current result.

7.3 Common Integration Problems

Missing header or library	Use search paths to the lib directory of one matching package.
Missing .so or symbol	Check runtime paths, complete dependencies, and ABI; do not mix mismatched packages.
Authorization failure	Confirm original key, GPU, version, validity, and full diagnostics.
Residual stagnation	Check matrix properties, initial guess, smoother, and iteration configuration; verify independently.

7.4 Numerical Accuracy and Residuals

$$r = b_{\text{original}} - A_{\text{original}} x, \qquad \rho = \frac{\|r\|_2}{\|b_{\text{original}}\|_2}.$$

Use the original system data. If $\|b\|_2 = 0$, report the absolute residual and state that the relative residual is undefined. With `rel_ini`, the library normalizes by the initial residual; for a nonzero initial guess it may differ from the RHS-normalized quantity above.

7.5 Finite Values and Display Precision

1. Check inputs, solution, residual entries, norms, and the final ratio for finiteness.
2. Use stable norms and higher accumulation precision when necessary to prevent overflow or underflow.
3. Report NaN/Inf explicitly, never as zero; use scientific notation for residuals.
4. Display timings with two decimal places and ms or s units, and keep counts integral; compare unformatted values for acceptance.

> **Note: Residual versus solution error**
>
> For ill-conditioned matrices, a small residual need not imply a small solution error. Compare with a reference solution when available. Record numerical acceptance failures separately from API execution failures.

7.6 Performance Measurement

$$T_{\text{solver}} = T_{\text{setup}} + T_{\text{solve}}.$$

This sum excludes file reading, conversion, transfers, creation, post-solve query, and independent verification. State end-to-end timing scope separately. Internal timings and synchronized wall-clock measurements may cover different work; do not mix them.

```
synchronize_device(); t0 = clock_now();
check(linglongamg_setup(solver, A));
synchronize_device(); t1 = clock_now();
check(linglongamg_solve(solver, &b, &x));
synchronize_device(); t2 = clock_now();
check(linglongamg_get_solve_info(solver, &info));
synchronize_device(); t3 = clock_now();
/* setup=t1-t0; solve=t2-t1; query_publish=t3-t2 */
```

Synchronization and clock functions are application pseudocode. Ensure GPU work completes before ending timing, not just command submission. Keep input, configuration, initial guess, tolerance, hardware, and timing boundaries identical in comparisons. Report initial runs, reused runs, and variability separately; retain timeouts and failures.

7.7 GPU Memory Planning

Hierarchy and workspace depend on matrix structure, coarsening, interpolation density, smoothing, and iteration space. Original nnz or file size cannot predict the peak. Measure it on the target device, reserve runtime capacity, and control concurrent allocations.

7.8 Issue Reports

Provide package name and checksum, header version, OS, GPU, driver, runtime, configuration, minimal program, first failing API, status, and original log. For matrix issues include dimensions, nnz, format, GPU memory, and original-equation residual; supply minimal input when shareable. Do not publish license contents or sensitive credentials.

Appendix A Example Input Files

This appendix describes CBD files accepted by the standard reader. The public API takes device CSR arrays; applications parse, convert, and validate business formats according to Section 4.

A.1 CBD Binary Layout

Order / field	Type and count
1 / nrows, ncols	One 32-bit integer each.
2 / nnz	One 64-bit integer.
3 / row_ptr	$n+1$ 64-bit integers.
4 / col_idx	nnz 32-bit integers.
5 / values	nnz FP64 real values.

The reader uses native binary layout; current platforms are little-endian without automatic byte swapping. Convert row pointers to API int32 and check overflow, final offset, sorting, and bounds. A filename extension is not format validation.

A.2 Sample RHS and Application Input

The standard sample generates $b = A\mathbf{1}$ and uses a zero initial guess. Applications prepare and upload their own b, x_0. Independent acceptance must preserve the original system; a generated sample RHS cannot replace the business RHS.

Appendix B Documentation and Support

Use documents matching the runtime. The manual source contains documentation and public-interface examples, not the numerical implementation or a valid license.

B.1 Package Documents

`lib/linglongamg.h`	Public C API declarations.
`README.md`	Platform requirements, steps, and restrictions.
`linglongamg_config.json`	Matching default configuration.
`test_src/`	Delivery sample source.
Release notes and checksums (if supplied)	Build changes, known issues, and file integrity.

B.2 Technical Support

Prepare the information in Section 7 and contact support through the WeCom code on the final page. Licenses are available at `https://store.solved-tech.com/`, also linked by a QR code on the final page. Section 2 explains license updates.

Appendix C Version History

The product version identifies the runtime, while the document revision identifies the guide. The following does not imply identical platform capabilities or performance.

C.1 Product Version

C.1.1 LinglongAMG 26.43

This guide covers the public CSR, Setup/Solve, and result-query interfaces. Consume licensed results only after a successful query. Consult delivery information for capabilities, runtime requirements, and compatibility. A document update does not imply a rebuilt or republished runtime.

C.2 Document Revisions

C.2.1 1.9

Reorganizes Chinese and English manuals into a consistent structure, expanding parameters, license lifecycle, phase dependencies, reuse, and validation. Distinguishes API status, internal convergence, and independent acceptance, and retains the current WeCom contact.

C.2.2 1.6

Introduced LinglongAMG-specific capabilities, input/output specifications, calling examples, and deployment workflows in a restrained technical-manual layout.

License portal

WeCom

store.solved-tech.com